# Como Ler e Interpretar os Relatórios de Segurança

## Projeto Final DevSecOps Clube Aurora (Turma 1746)

> Este guia ensina você a dissecar cada linha dos relatórios gerados pelas ferramentas de segurança do pipeline (**Gitleaks, Hadolint, Trivy FS, Semgrep SAST, Semgrep Compose, Trivy Image e OWASP ZAP**). Todos os exemplos citados abaixo são trechos reais extraídos das execuções do projeto em `logs_execucao/`.

---

## 1. Visão Geral: A Arquitetura dos Relatórios

No DevSecOps, ferramentas automatizadas não servem para "apontar o dedo", mas para fornecer telemetria estruturada. Os relatórios aparecem em três lugares no GitHub Actions:
1. **Console do Step (Logs de Execução):** Saída de texto ao vivo com tabelas formatadas e contadores;
2. **Resumo do Job (Job Summary):** Tabelas em Markdown renderizadas diretamente na interface web da execução;
3. **Aba Security (Code Scanning / SARIF):** Alertas integrados e rastreáveis vinculados aos arquivos do repositório.

---

## 2. Anatomia do Gitleaks (Detecção de Segredos)

O **Gitleaks** varre o repositório e o histórico Git à procura de senhas, chaves privadas e tokens de API expostos.

### Excerto Real do Log ( `logs_execucao/vulneravel/vuln_static.md` , linhas 244 a 254):

```
Finding:     ADMIN_INITIAL_TOKEN=eyJhZG1pbiI6dHJ1ZSwidXNlciI6ImFybWFuZG8ifQ.compromised
Secret:      eyJhZG1pbiI6dHJ1ZSwidXNlciI6ImFybWFuZG8ifQ.compromised
RuleID:      generic-api-key
Entropy:     4.780273
File:        /src/.env
Line:        7
Fingerprint: /src/.env:generic-api-key:7

6:23AM WRN leaks found: 1
##[error]Process completed with exit code 1.
```

### O que significa cada campo:

- `Finding` / `Secret` : O valor literal do segredo identificado no código.
- `RuleID` : O padrão de regex catalogado (ex: `generic-api-key` , `aws-access-token` ).
- `Entropy` : O nível de aleatoriedade dos caracteres (senhas e tokens criptográficos possuem alta entropia).
- `File` e `Line` : O arquivo e a linha onde o segredo foi gravado.

---

## 3. Anatomia do Trivy FS (SCA de Dependências)

O scanner **Trivy Filesystem** inspeciona os manifestos de pacotes (como `requirements.txt`). Quando encontra vulnerabilidades conhecidas (CVEs), ele gera uma tabela estruturada.

### Excerto Real do Log (`logs_execucao/vulneravel/vuln_static.md`, linhas 324 a 345):

```
requirements.txt (pip)
----------------------
Total: 10 (HIGH: 10, CRITICAL: 0)


 Library   Vulnerability   Severity  Status  Installed Version  Fixed Version

 gunicorn  CVE-2024-1135   HIGH      fixed   20.1.0             22.0.0          python-gunicorn: HTTP Requ
           CVE-2024-6827   HIGH      fixed   20.1.0             22.0.0          gunicorn: HTTP Request Smu
 urllib3   CVE-2023-43804  HIGH      fixed   1.26.5             2.0.6, 1.26.17  python-urllib3: Cookie req
 urllib3   CVE-2025-66418  HIGH      fixed   1.26.5             2.6.0           urllib3: Unbounded decompr
 urllib3   CVE-2025-66471  HIGH      fixed   1.26.5             2.6.0           urllib3: Streaming API imp
 urllib3   CVE-2026-21441  HIGH      fixed   1.26.5             2.6.3           urllib3: Safeguard bypass
 urllib3   CVE-2026-44431  HIGH      fixed   1.26.5             2.7.0           urllib3: Information disc
 urllib3   CVE-2026-97687  HIGH      fixed   1.26.5             2.8.0           urllib3: Traffic intercept
 urllib3   CVE-2026-97689  HIGH      fixed   1.26.5             2.8.0           urllib3: Denial of Service
 werkzeug  CVE-2024-34069  HIGH      fixed   2.2.3              3.0.3           werkzeug: Routing parse cr

##[error]Process completed with exit code 1.
```

### O que significa cada coluna:

- **`Library`:** Nome do pacote de software afetado (ex: `gunicorn`, `urllib3`, `werkzeug`).
- **`Vulnerability`:** O identificador universal da falha (**CVE - Common Vulnerabilities and Exposures**). Você pode pesquisar esse código no NVD (National Vulnerability Database) para entender a prova de conceito do exploit.
- **`Severity`:** Grau de severidade de acordo com a pontuação CVSS (`UNKNOWN`, `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`). O pipeline do Clube Aurora está configurado com gate em `HIGH,CRITICAL`.
- **`Status`:** Indica se existe correção disponível (`fixed`) ou se ainda não foi lançada (`affected`).
- **`Installed Version`:** A versão atualmente declarada no seu `requirements.txt`.
- **`Fixed Version`:** A versão mínima que você deve declarar para eliminar a vulnerabilidade.
- **`Title`:** Resumo descritivo da falha e link da documentação de segurança.

---

## 4. Anatomia do Semgrep SAST (Código-Fonte)

O **Semgrep** analisa a Árvore Sintática Abstrata (AST) do código Python sem executá-lo, procurando padrões que violem regras de segurança.

### Excerto Real do Log (`logs_execucao/vulneravel/vuln_static.md`, linhas 555 a 580):

```
 2 Code Findings


    /src/app/db.py
     python.lang.security.audit.formatted-sql-query.formatted-sql-query
          Detected possible formatted SQL query. Use parameterized queries instead.
          Details: https://sg.run/EkWw
          102 rows = get_db().execute(query).fetchall()
```

```
    python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
          Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in
          Injection. In order to execute raw query safely, prepared statement should be used.
          Details: https://sg.run/2b1L
          102 rows = get_db().execute(query).fetchall()
          ⋮

Ran 290 rules on 5 files: 2 findings.
##[error]Process completed with exit code 1.
```

### Como ler o apontamento do Semgrep:

1. **Arquivo e Linha (`/src/app/db.py:102`):** Aponta a localização exata no código onde a chamada perigosa foi executada.
2. **Nome da Regra (`rule: python.lang.security.audit.formatted-sql-query`):** Descreve a categoria do problema (neste caso, SQL formatado via string).
3. **Recomendação Curta:** *"Use parameterized queries instead"* (Use consultas parametrizadas).
4. **Link do Semgrep Registry (`https://sg.run/...`):** Oferece exemplos de código vulnerável vs. código seguro.
5. **Código de Saída (`exit code 1`):** Como foram encontrados 2 achados e o Semgrep rodou com `--error`, o processo encerra com erro, reprovando o job.

---

## 5. Anatomia do Hadolint (Linter de Dockerfile)

O **Hadolint** audita as instruções do `Dockerfile` contra as melhores práticas da comunidade Docker e regras da Open Container Initiative (OCI).

### Excerto Real do Log (`logs_execucao/vulneravel/vuln_static.md`, linha 187):

```
{
  "line": 6,
  "code": "DL3007",
  "message": "Using latest is prone to errors if the image will ever update. Pin the version explicitly to
  "column": 1,
  "file": "Dockerfile",
  "level": "warning"
}
```

### Elementos-chave do Hadolint:

- `code: DL3007`: Código catalogado da regra. O prefixo `DL` refere-se a regras de Dockerfile (`SC` refere-se a regras do ShellCheck para comandos `RUN`).
- `line: 6`: A linha no `Dockerfile` onde a violação ocorreu (no caso, a linha `FROM python:latest`).
- `level: warning`: Nível de criticidade (pode ser `error`, `warning`, `info` ou `style`). O pipeline monitora avisos e falhas para impedir imagens mutáveis em produção.

---

## 6. Anatomia do Semgrep Compose (Segurança de IaC)

O **Semgrep Compose** audita o arquivo `docker-compose.yml` verificando privilégios perigosos no host.

### Excerto Real do Log (`logs_execucao/vulneravel/vuln_static.md`, linhas 681 a 707):

```
 2 Code Findings


    /src/docker-compose.yml
     yaml.docker-compose.security.privileged-service.privileged-service
          Service 'aurora-api' is running in privileged mode. This grants the container the equivalent of
          capabilities on the host machine. Remove the 'privileged' key to disable this capability.
          7 privileged: true

     yaml.docker-compose.security.exposing-docker-socket-volume.exposing-docker-socket-volume
          Exposing host's Docker socket to containers via a volume. The owner of this socket is root. Givi
          someone access to it is equivalent to giving unrestricted root access to your host. Remove
          'docker.sock' from volumes to prevent this.
          15    - /var/run/docker.sock:/var/run/docker.sock

Ran 6 rules on 1 file: 2 findings.
##[error]Process completed with exit code 1.
```

---

## 7. Anatomia do Trivy Container Image Scan (Segurança da Imagem)

Diferente do Trivy FS que lê apenas o arquivo de texto `requirements.txt`, o **Trivy Image** disseca a imagem já construída no daemon do Docker. Ele inspeciona tanto as camadas da aplicação quanto os pacotes do sistema operacional Linux (pacotes `.deb` do Debian).

### Excerto Real do Log ( `logs_execucao/vulneravel/vuln_dynamic.md` , linhas 280 a 346):

```
clube-aurora-api:local (debian 13.7)
====================================
Total: 2 (HIGH: 2, CRITICAL: 0)

Python (python-pkg)
===================
Total: 12 (HIGH: 12, CRITICAL: 0)
##[error]Process completed with exit code 1.
```

Quando a imagem do contêiner possui vulnerabilidades acima do limiar definido pelo Quality Gate ( `--severity HIGH,CRITICAL --exit-code 1` ), o comando termina com código de erro `1` .

---

## 8. Anatomia do OWASP ZAP Baseline DAST (Análise Dinâmica de Aplicação)

O **OWASP ZAP Baseline Scan** analisa a aplicação viva via requisições HTTP na porta 8080.

### 8.1 Excerto da Versão Vulnerável ( `logs_execucao/vulneravel/vuln_dynamic.md` , linhas 532 a 541):

```
FAIL-NEW: X-Content-Type-Options Header Missing [10021] x 4
    http://localhost:8080 (200 OK)
    http://localhost:8080/ (200 OK)
    http://localhost:8080/robots.txt (200 OK)
    http://localhost:8080/sitemap.xml (200 OK)
FAIL-NEW: Content Security Policy (CSP) Header Not Set [10038] x 1
    http://localhost:8080/admin (404 Not Found)
FAIL-NEW: 2 FAIL-INPROG: 0  WARN-NEW: 0 WARN-INPROG: 0  INFO: 0 IGNORE: 0   PASS: 59
##[error]Scan action failed as ZAP has identified alerts. Error: The process '/usr/bin/docker' failed with
```

```
```

O ZAP identifica que as rotas retornam 200 OK mas não incluem os cabeçalhos de segurança obrigatórios declarados no arquivo `.zap/rules.tsv`. Por isso, as regras disparam como `FAIL-NEW: 2` e reprovam o Quality Gate com código 1.

### 8.2 Excerto da Versão Corrigida (`logs_execucao/corrigido/fixed_dynamic.md`, linha 593):

```
FAIL-NEW: 0 FAIL-INPROG: 0  WARN-NEW: 0 WARN-INPROG: 0  INFO: 0 IGNORE: 0   PASS: 61
```

- `PASS: 61`: O ZAP executou 61 regras de segurança passiva contra as rotas da aplicação Flask e todas foram aprovadas com sucesso.
- `FAIL-NEW: 0`: Nenhuma regra falhou em nível de erro.
- `WARN-NEW: 0`: Não houve alertas de advertência (graças aos cabeçalhos de segurança implementados em **F5**).
- `zap-report.html`: O arquivo HTML detalhado fica disponível para download na seção de **Artifacts** da execução.